

PLAN DE ITERACIÓN SPRINT 02

DOCUZEN CONSTRUCCIÓN DE SOFTWARE I

**Ingesta Segura y Directorio Cifrado
RF-04 y RF-05**

Captura TWAIN/WIA, directorio temporal cifrado y tolerancia a atascos

Control de Versiones

Versión	Fecha	Autor	Descripción
1.0	12/09/2026	Carlos Ayala y Ahmed Akhtar	Plan de Iteración Sprint 02 basado en Product Backlog y trazabilidad.
1.1	12/09/2026	Carlos Ayala y Ahmed Akhtar	Se incorporó captura del código fuente implementado en el Sprint 02.

1. Información General

Los datos de la iteración y del equipo se registran en la siguiente tabla.

Curso	Construcción de Software I
Iteración	Sprint 02
Duración planificada	Ciclo semanal
Fecha de entrega	18/05/2026
Líder	Carlos Daniel Ayala Ramos
Equipo	Carlos Daniel Ayala Ramos, líder; Ahmed Hasan Akhtar Oviedo, integrante

2. Objetivo de la Iteración

Implementar RF-04 y RF-05 para una captura segura mediante TWAIN/WIA, directorio temporal AES-256 y recuperación ante atascos.

3. Alcance

Incluye SRS/SAD, integración TWAIN/WIA, directorio cifrado, pausa/reanudación, build Windows y 12 pruebas del sprint.

4. Entregables Esperados

Los productos verificables de la iteración se detallan en la siguiente tabla.

Entregable	Descripción	Aceptado
Especificación	SRS: CU-RF04 y JAM_DETECTED	Sí
Diseño	SAD: captura TWAIN y SecureFileWriter	Sí
Implementación	TWAIN/WIA, AES-256 y recuperación	Sí
Validación y trazabilidad	12 pruebas y evidencia Windows 10/11	Sí

5. Cronograma y Actividades

Las actividades se ejecutaron durante el ciclo semanal y se cerraron en la fecha de entrega registrada.

ID	Actividad	Responsable	Inicio	Fin	Estado
A1	Documentar RF-04/RF-05 y fallos	Carlos Ayala	Ciclo semanal	18/05/2026	Completado
A2	Diseñar captura y recuperación segura	Equipo	Ciclo semanal	18/05/2026	Completado

ID	Actividad	Responsable	Inicio	Fin	Estado
A3	Implementar TWAIN/WIA y directorio cifrado	Ahmed Akhtar	Ciclo semanal	18/05/2026	Completado
A4	Ejecutar 12 pruebas del sprint	Equipo	Ciclo semanal	18/05/2026	Completado
A5	Publicar build y registrar evidencias	Equipo	Ciclo semanal	18/05/2026	Completado

6. Recursos Requeridos

Cliente Edge Windows 10/11, drivers TWAIN/WIA, escáneres de prueba, AES-256, repositorio, VPS, SRS/SAD y catálogo.

7. Riesgos y Mitigaciones

Los riesgos identificados y sus acciones de mitigación se registran en la siguiente tabla.

Riesgo	Impacto	Mitigación
Drivers TWAIN/WIA incompatibles	Alto	Probar distintos controladores y mantener fallback.
Permisos del directorio cifrado	Medio	Validar rutas y permisos por sesión.
Pérdida tras atasco	Alto	Conservar índice y reanudar en máximo 5 s.

8. Criterios de Aceptación

Sin archivos en claro; TWAIN/WIA operativo; JAM_DETECTED pausa y reanuda en ≤ 5 s sin pérdida/duplicación; 12 pruebas completadas.

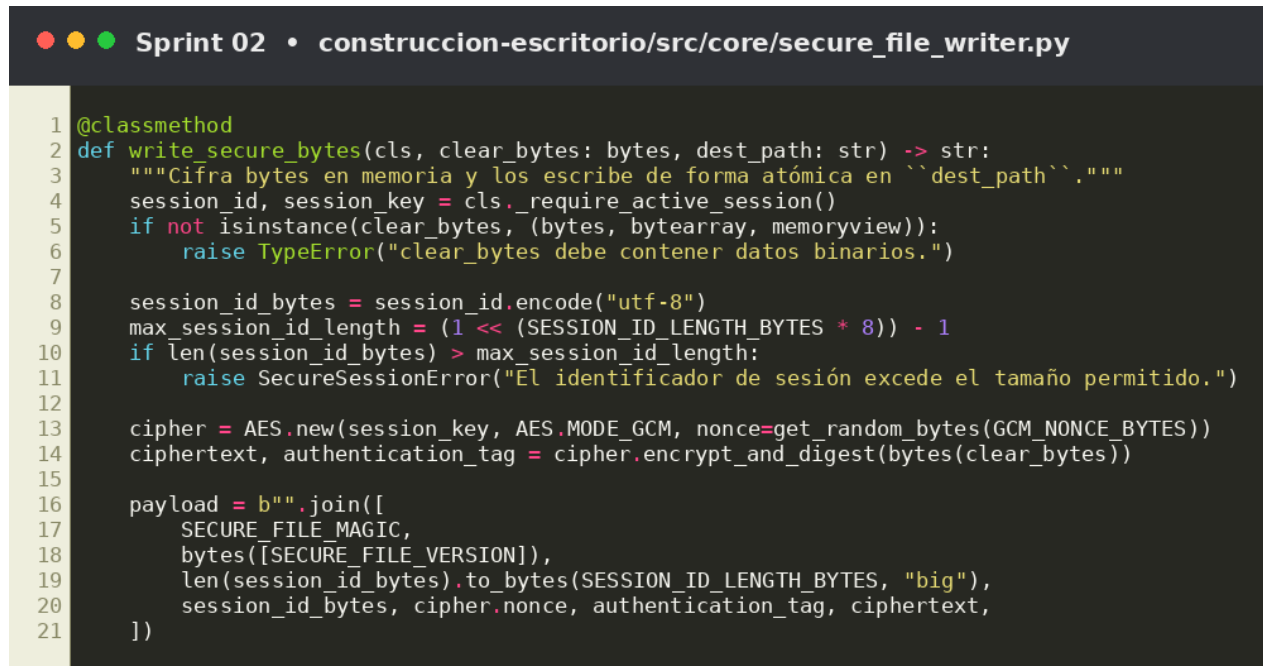
9. Evidencias

Las evidencias verificables del sprint se registran en la siguiente tabla.

Evidencia	Ubicación	Responsable
Product Backlog	Sprint 02 — 18/05/2026	Equipo
Documentación	SRS/SAD de ingesta segura	Carlos Ayala
Pruebas	12 casos del Sprint 02	Ahmed Akhtar
Build	Cliente Windows de prueba	Equipo
Código fuente	construccion-escritorio/src/core/secure_file_writer.py	Equipo

9.1 Evidencia de código fuente implementado

La evidencia muestra el bloque encargado de cifrar los bytes de cada folio antes de persistirlos en el directorio temporal. Se utiliza una clave de sesión AES-256 y autenticación GCM, evitando que el documento permanezca en claro durante la ingesta.



```
Sprint 02 • construccion-escritorio/src/core/secure_file_writer.py

1 @classmethod
2 def write_secure_bytes(cls, clear_bytes: bytes, dest_path: str) -> str:
3     """Cifra bytes en memoria y los escribe de forma atómica en ``dest_path``."""
4     session_id, session_key = cls._require_active_session()
5     if not isinstance(clear_bytes, (bytes, bytearray, memoryview)):
6         raise TypeError("clear_bytes debe contener datos binarios.")
7
8     session_id_bytes = session_id.encode("utf-8")
9     max_session_id_length = (1 << (SESSION_ID_LENGTH_BYTES * 8)) - 1
10    if len(session_id_bytes) > max_session_id_length:
11        raise SecureSessionError("El identificador de sesión excede el tamaño permitido.")
12
13    cipher = AES.new(session_key, AES.MODE_GCM, nonce=get_random_bytes(GCM_NONCE_BYTES))
14    ciphertext, authentication_tag = cipher.encrypt_and_digest(bytes(clear_bytes))
15
16    payload = b"".join([
17        SECURE_FILE_MAGIC,
18        bytes([SECURE_FILE_VERSION]),
19        len(session_id_bytes).to_bytes(SESSION_ID_LENGTH_BYTES, "big"),
20        session_id_bytes, cipher.nonce, authentication_tag, ciphertext,
21    ])
```

Figura 2. Cifrado AES-256-GCM del almacenamiento temporal de la sesión (RF-04). Fuente: código fuente del proyecto DocuZen (construccion-escritorio/src/core/secure_file_writer.py).

10. Indicadores de Éxito

Los indicadores reflejan el resultado registrado al cierre de la iteración.

Indicador	Meta	Resultado	Estado
Requerimientos del sprint	RF-04 y RF-05 completados	2 RF	Conforme
Recuperación	Reanudación ≤ 5 s	≤ 5 s	Conforme
Pruebas ejecutadas	12 casos planificados	12 casos	Conforme
Interoperabilidad	Captura TWAIN/WIA validada	Validada	Conforme

11. Seguimiento y Control

El cierre confirmó captura segura y recuperación ante atascos. Se acordó continuar con OCR, renombrado y Bandeja de Pendientes.

12. Lecciones Aprendidas y Cierre

Probar hardware temprano reduce sorpresas de integración. Separar estado de cola y dispositivo facilita una recuperación confiable.